

專案功能現況總覽 (Project Capability Map)

- **非對稱條帶化 RAID 引擎**：第二版完整實作了專用的「非對稱 RAID 0」演算法，根據每顆硬碟的實測速度比率來分配條帶寬度。速度較快的 SSD 獲得更大的寫入區塊，較慢的硬碟獲得較小區塊，使各硬碟幾乎同時完成請求，提高整體吞吐率。條帶引擎支援動態計算多階段 (Phase) 的 LBA 映射，由 `stripe_engine` 模組實現。
- **掃描效能優化 (O_DIRECT 繞過系統快取)**：`disk_scanner` 和 `bench_io` 等模組在掃描或基準測試時使用原始 I/O 模式 (透過 `CreateFileW` 搭配 `FILE_FLAG_NO_BUFFERING` 等旗標) 直接讀寫磁碟，繞過作業系統快取。此作法能準確衡量硬碟吞吐性能，特別適用於混合 NVMe/SATA 的環境下測試真實速度。
- **4GB 記憶體快取與 Dirty Page Bitmap 刷寫**：在 `ram_cache` 模組中，系統預設使用 4GB 大小的 RAM Cache (`CACHE_DEFAULT_MB = 4096` MB)，以 64KB 為單位快取寫入資料。設計了髒區塊位圖 (dirty bitmap) 來追蹤哪些區塊已修改。主執行緒寫入時將資料拷貝到快取，同時設定對應位圖位元；背景線程則不斷掃描該位圖，找出連續髒區塊並進行寫入，成功刷寫後清除相應位元。這種雙執行緒配合機制既確保了高速寫入性能，也保證資料能最終落盤。
- **WinFsp FUSE 本機虛擬磁碟掛載 (Drive Type = 固定磁碟)**：利用 `fuse_bridge` 模組整合 WinFsp，將 RAID 卷掛載為本機固定磁碟。FUSE Bridge 實作了必要的檔案系統回呼函式 (如 `statfs`, `truncate`, `open`, `read`, `write` 等)，使 Windows 能以類似本機硬碟的方式讀寫該虛擬卷。在 `statfs` 回呼中，根據條帶卷大小和已寫入資料計算可用空間；在 `truncate` 回呼中更新開啟檔案的大小。此設計允許在 Windows 環境下以類似本機磁碟的方式使用虛擬 RAID 卷。
- **JSON 設定與狀態持久化**：系統會將用戶設定 (例如條帶單元大小、緩存容量、掛載磁碟代號等) 寫入 `%APPDATA%\RAIDTEST\config.json`，使用 `fwprintf` 逐行輸出 JSON 格式內容；啟動時透過 `config_load` 讀取並用 `swscanf` 解析回原有結構。此外，`superblock` 模組可將 RAID 組態資訊 (磁碟清單、讀寫性能、條帶階段配置等) 寫入各磁碟的 `RAIDTEST\superblock.dat` 檔，支援啟動時根據先前狀態恢復卷結構。這套持久化循環讓使用者可在重啟後「還原」先前的 RAID 配置。
- **使用者介面與執行緒同步**：第二版加入了簡易 GUI 介面。在圖形模式下，`gui` 模組建立 Windows 對話框，顯示可用磁碟和當前配置狀態。使用者在 GUI 上觸發掃描、初始化、建立卷等操作時，實際透過背景執行緒 (`action_worker`) 依序執行對應的命令。兩者之間透過全域狀態鎖和訊息機制協調，例如：Daemon 模式下捕捉 `Ctrl+C` 事件觸發 `g_daemon_stop` 事件，通知主執行緒卸載並關閉；GUI 模式則更新控制項來顯示進度和日誌。這確保了多執行緒下使用者操作、卷管理和背景同步刷寫等工作能和諧運行。

核心架構與實作方法解析 (Implementation Architecture)

1. **條帶映射演算法**：`stripe_engine` 模組實現了非對稱條帶化引擎。系統先透過 `stripe_volume_normalize_ratios` 根據每顆磁碟的寫入速度計算相對權重。`stripe_volume_create` 接著為每個階段 (Phase) 設置參與磁碟數量、對應的起始位址和大小。對於每筆 I/O 請求，核心函式 `stripe_volume_map_lba` 迴圈遍歷所有階段，找出該邏輯偏移量 (Virtual Offset) 所屬的階段。如果該階段只有一顆磁碟活躍，則直接映射到該磁碟的物理偏移；若多磁碟並行，則將偏移量對應到一個「循環區段」，並根據各磁碟的權重累積落點來決定實際寫入的磁碟和偏移。具體來說，`map_single_byte` 函式先計算 `cycle_index = offset_in_phase / cycle_size`，再遍歷

每個活躍磁碟累加其配置的條帶大小 (`ratios[j]`)，直到找到屬於哪顆磁碟的區段；然後計算該磁碟上的物理偏移量。整體而言，映射邏輯確保了資料按照磁碟速度分配，並在完成寫入時保持所有磁碟同步完成請求。

2. **4GB 快取與雙執行緒同步**：快取系統分為兩個主要部分：主線程寫入和背景 flush。`ram_cache` 初始化時按 64KB 區塊算出總塊數並分配位圖，`EnterCriticalSection(&lock)` 保護關鍵操作。寫入時，`cache_write` 首先拿鎖，將資料複製進緩衝區 `cache->buffer`，並對應地在 `cache->dirty_map` 中將涉及的位元設為 1，然後釋放鎖。刷新時，`cache_flush_thread` 週期性執行。它首先在鎖內掃描整個位圖統計髒塊數。若超過閾值，它會進入 `cache_flush_all`，在一個大迴圈中找出連續的髒塊段。在每次找到髒塊起點後，在鎖內計算該區段長度並清除對應位元，然後釋放鎖，再以 `stripe_volume_write` 將該段資料寫入磁碟。這種「鎖內搜索+鎖外寫入」設計能避免長時間鎖住整個位圖，讓主線程繼續快取後續資料。完成寫入後，如有剩餘髒塊會重複上述步驟。寫入結束或達到批次上限後，flush 線程依髒塊百分比動態調整休眠時間，再次進行下一輪檢查。該結構保證了快取資料不丟失且能持續推送到硬碟，同時最大化並行度。
3. **JSON 設定/狀態讀寫**：`config.c` 手動處理 JSON 格式。`config_save` 生成配置檔路徑（位於 `%APPDATA%\RAIDTEST\config.json`），並以 UTF-8 格式開檔，用 `fwprintf` 寫出每項屬性，如 `"version":`，`"cache_mb":`，`"mount_letter":` 等；最後關閉檔案。`config_load` 先填充預設值，再嘗試打開同一路徑；如果檔案存在，就逐行讀取 (`fgetws`)，查找關鍵字並用 `swscanf` 抽取數值，恢復到 `APP_CONFIG` 結構。此設計簡潔輕量，但要求 JSON 格式嚴格。配置讀取後，程式可在 `main` 中自動執行命令（例如組成 `"select ..."`，`"init ..."`，`"create"`，`"cache"`，`"mount"` 等一系列命令）來還原 RAID 結構。由此可見，設定持久化不僅限於單純參數保存，也串聯到內部指令流程，使系統能根據保存的狀態重建條帶卷。
4. **WinFsp FUSE 掛載流程**：在掛載流程中，若選擇 GUI 模式，會直接啟動 WinFsp 的 `fuse_main` 事件迴圈；Daemon 模式則在新執行緒中呼叫 `fuse_mount_volume`，並同時保持主執行緒監聽輸入（例如支援從控制台 `exit` 指令結束）。FUSE 回呼函式如 `open`，`read`，`write` 等均直接對應到條帶讀寫：比如 `write` 會讀取使用者緩衝區然後呼叫 `stripe_volume_write`，成功後更新全域變數 `g_ctx.highest_byte_written`（反映最大寫入偏移）。`truncate` 則搜尋開啟檔案列表並調整其大小；`statfs` 基於當前配置計算總塊數和已用塊數（`f_bsize`，`f_blocks`，`f_bfree` 等）。整個過程中，WinFsp 提供的是 FUSE 統一介面，而後端則有我們的 RAID 引擎處理實際 I/O，前者負責將 Windows 的檔案操作轉換為對條帶系統的存取。
5. **介面與同步協調**：`gui.c` 定義了基本的 Windows 對話框介面，包括磁碟列表、狀態欄和日誌視窗。當使用者按下「Scan」「Init」「Create」等按鈕時，對話框回呼會創建並傳遞任務到 `action_worker` 執行緒。該執行緒實際上會呼叫 `cmd_process` 來執行對應的文字命令序列；完成後透過同步機制（例如 `PostMessage` 或全局狀態變數）將結果傳回給主線程以更新 GUI。Daemon 模式則利用 `SetConsoleCtrlHandler` 註冊控制訊號，捕捉 `CTRL_C_EVENT` 等，並在發生時設定事件物件 `g_daemon_stop`。主執行緒在事件迴圈中等待此事件，一旦觸發便執行 `fuse_unmount_volume` 和清理操作。整體而言，GUI 與底層驅動用鎖、事件和訊息確保無競爭，避免多執行緒導致資源衝突。

系統隱患、死角與安全漏洞掃描 (Vulnerability & Weakness Scan)

- **Statfs/Truncate 空間更新不一致**：目前 `statfs` 依賴 `g_ctx.highest_byte_written` 來計算已用空間，但 `truncate` 回呼只改變開啟檔案的 `size` 而不更新這個全域變數。若檔案被截短或刪除，`highest_byte_written` 會維持舊值，導致 `statfs` 報告的可用空間過少。換言之，執行檔案縮減後，用戶端並不會立即「看見」釋放的空間；需要重啟或手動同步才會生效。此外，若多檔案同時寫入或釋放空間，這種全域計數方式可能導致競態條件，隱藏真實可用空間。

- **高併發寫入下的快取競爭與一致性風險**：雖然快取採用臨界區來控制位圖，但仍有可能發生競爭問題。例如，主線程可能在 `memcpy` 資料到 `cache->buffer` 時進行，而此時 `flush` 線程在另一區塊清除位圖後正在寫入。考慮極端情況：快取區域 A 在 `flush` 線程拿完 `lock` 後被清除位元，接著主線程立即寫入新資料並重新設為髒位，然後 `flush` 線程才實際寫入剛才在清除前複製出來的舊資料。這會造成「舊資料」錯誤地覆蓋磁碟，導致資料不一致。雖不直接導致緩衝區溢位（因為大小受限），但可能出現邏輯錯誤，寫出的資料並非最新內容。
- **單一硬碟 Superblock 缺乏容錯**：`superblock_write` 中迴圈產生的目的是寫入每個磁碟，但實作上卻只將超級區塊檔寫到最後一顆磁碟。換句話說，所有 RAID 組態資訊都被儲存在單一物理碟上。如果該磁碟故障或該檔案損壞，整個 RAID 組態將無法恢復。缺乏冗餘寫入和檢查機制，讓系統容易在硬體失效時「一蹶不振」。理想做法應在每顆磁碟都留存副本或使用多重備份，並在載入時檢查 CRC、版本號，以選擇正確的配置。但當前實現並未處理多副本或異常讀取情況。
- **Daemon 模式優雅退出風險**：雖然已實作控制事件處理，在收到中斷訊號時會嘗試卸載並清理，但若進程被強制終止（例如直接結束程序、Winfsp 未卸載等），快取裡尚未寫入的資料將丟失，且系統可能掛在未卸載狀態。特別是 `cache_flush_thread` 未明確監聽退出事件，只依賴 `cache->running`；若程式崩潰或不執行結束邏輯，寫入線程可能尚在運行，導致資源未釋放。此外，解除掛載（`fuse_unmount_volume`）的失敗也沒有重試策略，可能使虛擬磁碟保持鎖定狀態。總之，目前缺少確保「所有資料確實落盤再退出」的機制，存在資料遺失和系統不一致的風險。
- **其他效能瓶頸與潛在問題**：條帶映射與快取刷寫均使用大量迴圈操作，對於極大資料量可能造成 CPU 負荷。`cache_flush_all` 在計算髒塊時是一個 $O(N)$ 操作；`stripe_volume_map_lba` 內部多重迴圈在高速寫入下可能成為瓶頸。另外，FUSE 層的開啟檔案數限制（`g_open_files` 最大 64 個）也較低，可能限制並發度。`config_load` 的 JSON 解析相對原始，缺乏錯誤容忍；錯誤格式可能導致載入失敗而使用預設值，沒有嚴謹的使用者通知或日誌。這些雖屬實作細節，但在壓力下有可能引發穩定性或性能問題。

具體的代碼改進與修改方案 (Refactoring & Action Plan)

- **同步更新 Statfs/Truncate 資訊**：在 `fuse_bridge.c` 的 `truncate` 回呼中，除了更新開啟檔案的 `size`，也要同步調整 `g_ctx.highest_byte_written`。具體做法是在截斷後找出所有開啟檔案的最大尺寸，將 `highest_byte_written` 設為該值，或在截斷縮小檔案時重新計算最高值。例如：

```
int idx = find_open_file(path+1);
if (idx >= 0) {
    uint64_t oldSize = g_open_files[idx].size;
    g_open_files[idx].size = (uint64_t)size;
    if ((uint64_t)size < g_ctx.highest_byte_written) {
        // 重新計算目前開啟檔案的最大大小
        uint64_t maxSize = 0;
        for (int i = 0; i < g_open_count; i++) {
            if (g_open_files[i].size > maxSize)
                maxSize = g_open_files[i].size;
        }
        g_ctx.highest_byte_written = maxSize;
    } else {
        g_ctx.highest_byte_written = (uint64_t)size;
    }
}
```

此外，可在檔案關閉時（release 回呼）再次更新 `highest_byte_written`，避免長時間錯誤累積。這樣修改後，`stats` 報表將即時反映截斷或刪除操作導致的空間變化。

- **強化快取寫入與 Flush 同步**：可採用更嚴謹的鎖和緩衝策略。舉例來說，**鎖住資料複製階段**：在 `cache_flush_all` 中，當找到鱗塊時在鎖內先複製這段資料到臨時緩衝區，並清除位圖，然後再釋放鎖；最後使用複製出的資料執行寫入，如：

```
EnterCriticalSection(&cache->lock);
memcpy(tmp_buffer, cache->buffer + block_offset, total_len);
for (uint32_t i = b; i < b+run; i++)
    cache->dirty_map[i/8] &= ~(1 << (i%8));
LeaveCriticalSection(&cache->lock);
// 使用 tmp_buffer 進行寫入，避免與主線程資料衝突
stripe_volume_write(vol, tmp_buffer, block_offset, total_len);
```

這可避免當寫入進行時快取被修改。此外，在多核心平台上可加上記憶體屏障（MemoryBarrier）或使用原子操作更新位圖，確保所有處理器對資料狀態的一致視圖。這種重構確保主線程在 flush 過程中若再次寫入資料時，被寫入的資料和位圖狀態不會被誤刷到磁碟上。

- **實作 Superblock 冗餘寫入**：將超級區塊寫入所有物理磁碟，以容忍單碟故障。修改 `superblock_write` 如：

```
for (uint32_t i = 0; i < vol->disk_count; i++) {
    DISK_INFO* disk = vol->disks[i];
    wchar_t root[4] = { disk->drive_letter[0], L':', L'\\', 0 };
    StringCchPrintfW(dir_path, MAX_PATH, L"%ls%s", root, CONFIG_DIR);
    CreateDirectoryW(dir_path, NULL);
    StringCchPrintfW(tmp_path, MAX_PATH, L"%ls\\%ls.tmp", dir_path,
        SUPERBLOCK_FILENAME);
    StringCchPrintfW(final_path, MAX_PATH, L"%ls\\%ls", dir_path,
        SUPERBLOCK_FILENAME);
    HANDLE h = CreateFileW(tmp_path, GENERIC_WRITE, FILE_SHARE_READ, NULL,
        CREATE_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);
    if (h != INVALID_HANDLE_VALUE) {
        DWORD written;
        WriteFile(h, &sb, sizeof(sb), &written, NULL);
        FlushFileBuffers(h);
        CloseHandle(h);
        MoveFileExW(tmp_path, final_path, MOVEFILE_REPLACE_EXISTING);
    }
}
```

此外，`superblock_load` 應迴圈嘗試讀取每個磁碟的 `superblock.dat`，並對比其中的 `magic` 和 `version`，選擇一份完整且最新的作為恢復來源。如果檢測到 CRC 不符或大小錯誤，則跳過該副本。如此在某顆磁碟失效時，其它磁碟仍能提供 RAID 組態資訊，顯著提高容錯性。

- **優化 Daemon 模式退出流程**：在收到退出訊號時，首先通知快取線程停止並等待其結束，再執行卸載操作。修改步驟如下：收到 `g_daemon_stop` 事件後，設定 `g_state.volume.cache.running = false;`，然後呼叫 `WaitForSingleObject(flush_thread_handle, INFINITE);` 等待快取線程退出。之後呼叫一次 `cache_flush_all(&volume.cache, &volume)` 確保所有髒資料落盤。最後再呼叫 `fuse_unmount_volume` 卸載卷並清理資源。需要在 `daemon_start` 或 `daemon_main` 保存 `flush_thread_handle` 來等待線程結束。這樣做可避免程式在磁碟 I/O 尚未完成時就強制關閉，確保資料一致性。
 - **開發分工與後續計畫**：根據以上檢測結果，建議在下一階段（Phase 4.2）依模組分工重構與測試：**（1）快取與檔案系統小組**：負責快取同步邏輯重構與 `statfs` / `truncate` 修正；**（2）條帶引擎與超級區塊小組**：實作映射演算法優化、超級區塊冗餘寫入；**（3）系統 I/O 驗證小組**：加強磁碟掃描、錯誤檢查與日誌；**（4）整合測試小組**：開發自動化腳本測試高併發與硬碟故障場景。每組各自制定里程碑任務，並預留充分的單元與集成測試資源，確保重構後系統能達到生產等級的穩定性和效能。
-